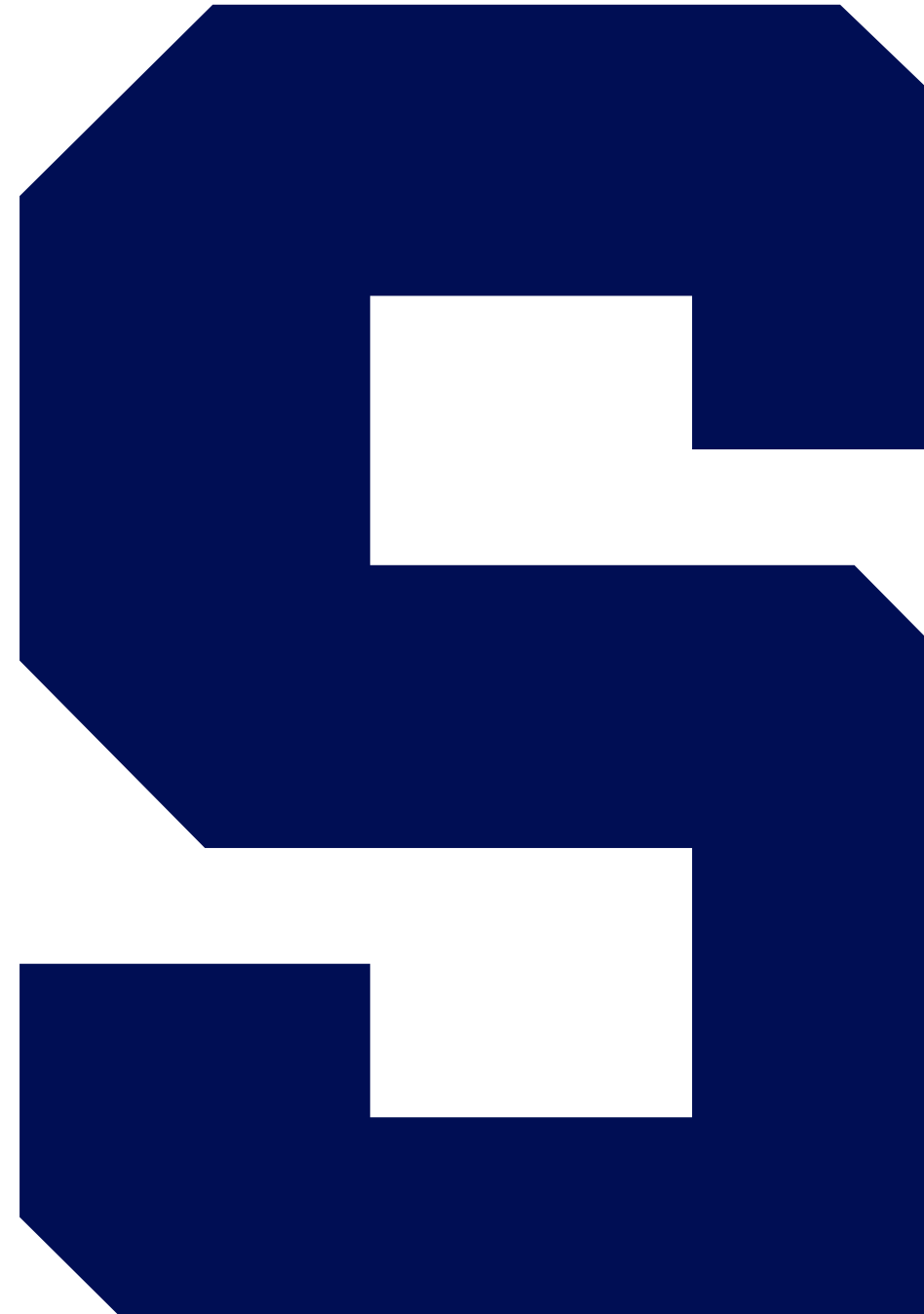


Logistic Regression



Logistic Regression

- A supervised learning algorithm used for **binary classification** — e.g., positive/negative sentiment, disease/no disease.
 - Estimates the probability that an instance belongs to a particular category.
 - Assumes a **linear relationship** between the log-odds of the dependent variable and the independent variables. Especially suited for problems with a linear decision boundary between the two classes.
 - Models this relationship using the **logistic function**.
 - Uses **binary cross-entropy** as its loss function.
-

The Logistic Function

$$p(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n)}}$$

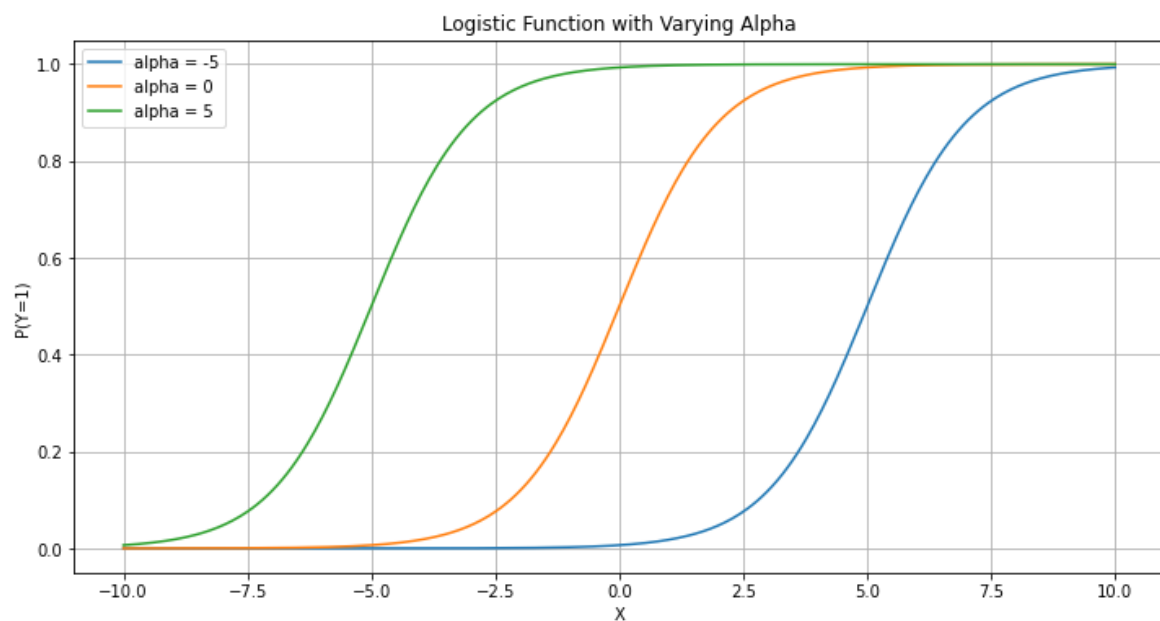
- $p(Y=1)$ is the predicted probability that the target variable Y belongs to class 1.
- e is the base of the natural logarithm.
- $\beta_0, \beta_1, \dots, \beta_n$ are the coefficients of the model, where β_0 is the intercept term, and $\beta_1, \dots, \beta_n$ are the coefficients for the respective features $X_1, \dots, X_n$.

Logistic function general form:

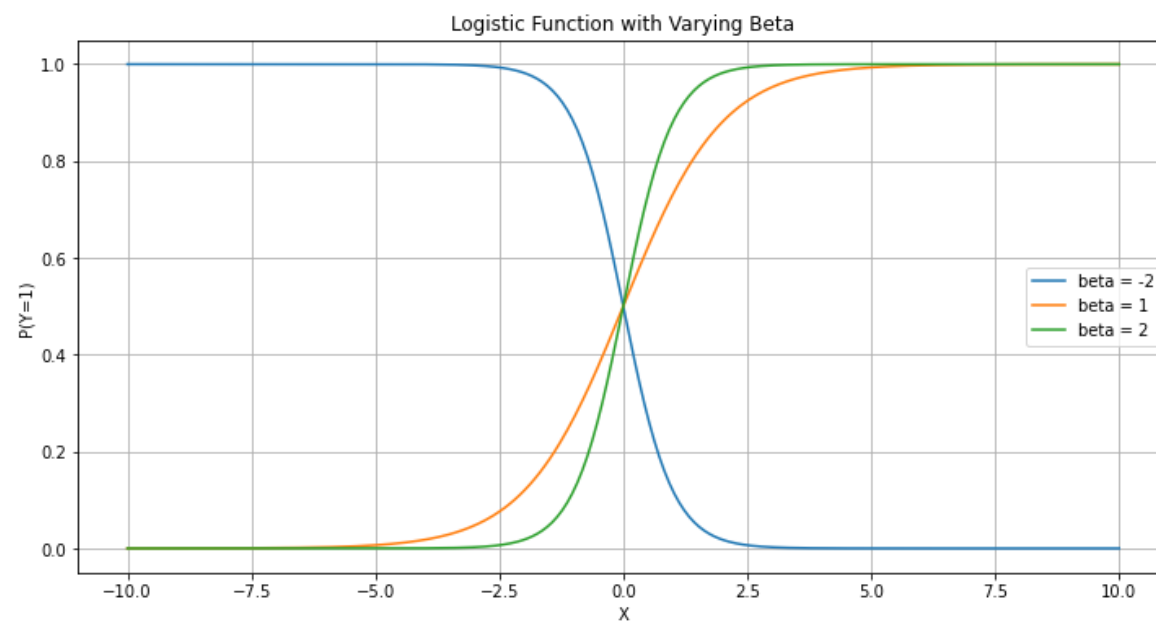
$$p(X) = \frac{1}{1 + e^{-(\alpha + \beta X)}}$$

Understanding the Logistic Function

$$P(Y = 1) = \frac{1}{1 + e^{-(\alpha + \beta X)}}$$



Alpha varies position



Beta varies slope

The logistic function squashes data to the range [0-1]

Binary Cross-Entropy

- **Binary cross-entropy loss** (a.k.a. log loss) measures the performance of a classification model whose output is a probability p whose value is between 0 and 1.

$$L(y, p) = -y \log(p) - (1 - y) \log(1 - p)$$

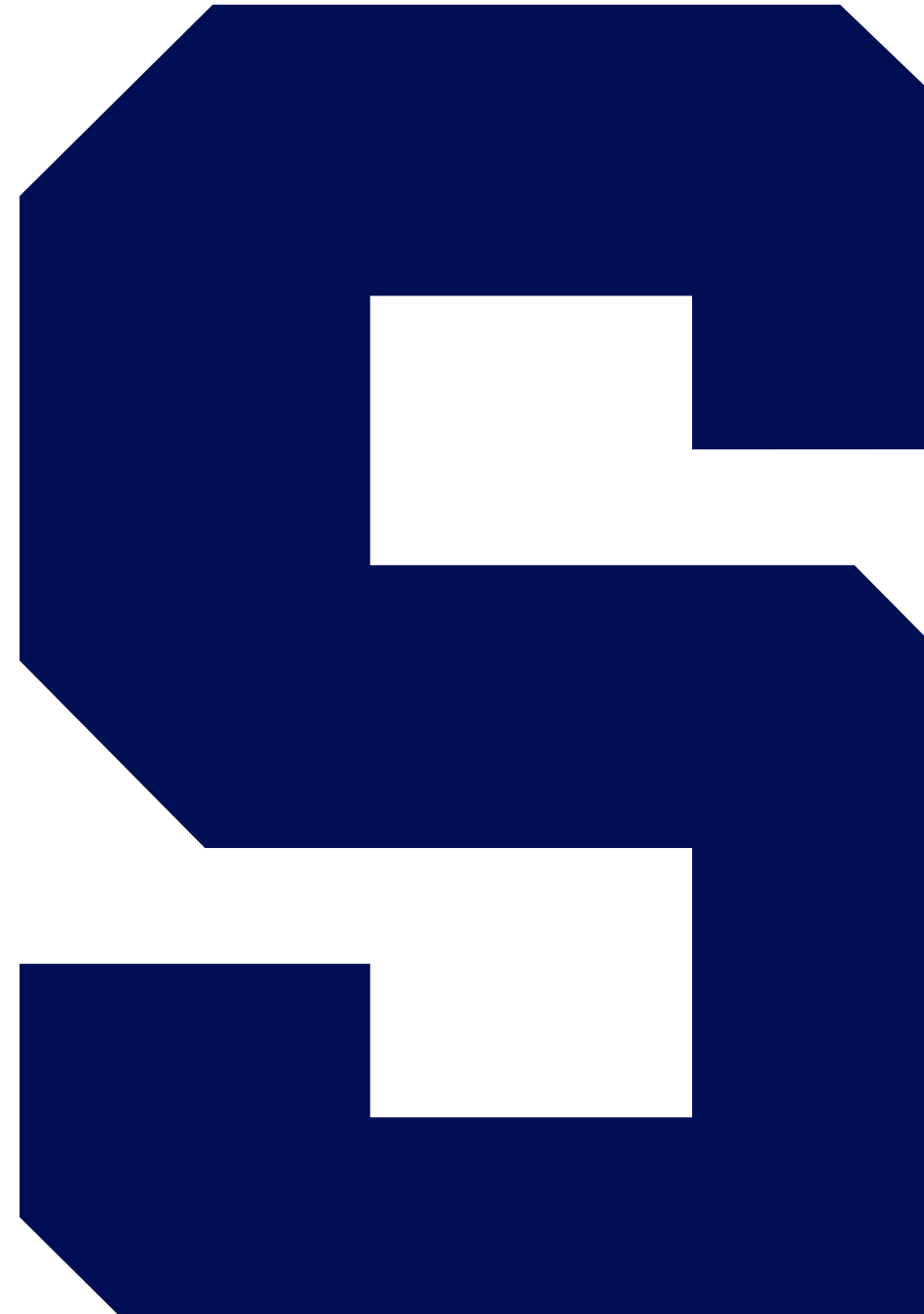
- **When the true label y is 1:**
 - The loss becomes $L = -\log(p)$.
 - If our model predicts $p = 1$ (perfect prediction), the loss is $L = 0$.
 - If our model predicts $p = 0$ (completely wrong), the loss tends to infinity.
- **When the true label y is 0:**
 - The loss becomes $L = -\log(1 - p)$.
 - If our model predicts $p = 0$ (perfect prediction), the loss is $L = 0$.
 - If our model predicts $p = 1$ (completely wrong), the loss tends to infinity.

The goal of logistic regression is to find the set of coefficients β that minimize this loss function.
Binary cross-entropy loss heavily penalizes predictions that are confidently wrong.

The Logistic Regression Algorithm

- 1. Initialization:** The model coefficients (β) are usually initialized to 0 or a small random number.
 - 2. Model Training:** During training, logistic regression uses an optimization algorithm (like gradient descent) to minimize the loss function. This involves making predictions using the current coefficients, calculating the error by comparing the predicted probabilities with the actual class labels, and then adjusting the coefficients based on this error.
 - 3. Updating Coefficients:** The coefficients are updated in a direction that reduces the loss.
 - 4. Convergence:** The process continues iteratively until the model converges, meaning the reduction in the loss function becomes negligible with further iterations, or until a specified number of iterations is reached.
-

KNN



Supervised Vs. Unsupervised Learning

Machine Learning (ML) is a subset of artificial intelligence (AI) where the primary goal is for computers to learn from data without being explicitly programmed. Generally, we can categorize machine learning tasks into two main types:

- **Supervised Learning:** The model is trained on a **labeled** dataset, which means for every input data, the correct result (or output) is known. Once trained, the model can start making predictions on new, unseen data.
 - **Unsupervised Learning:** The model is trained on an **unlabeled** dataset, trying to learn the underlying structures from the data, such as grouping or clustering similar data points together.
-

Model-based Vs. Instance-based Learning

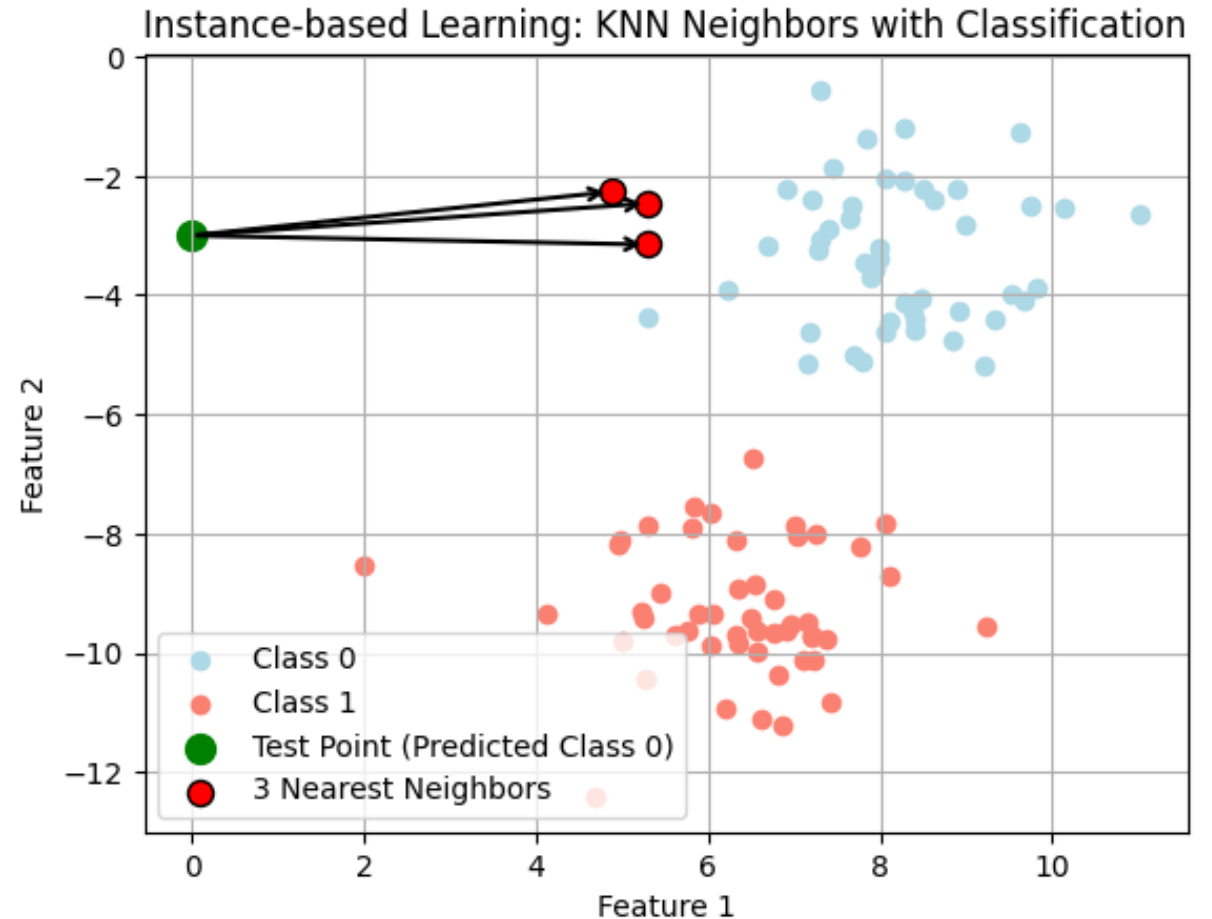
Within supervised learning, we can distinguish between model-based and instance-based learning, as follows.

- **Model-based Learning:** An algorithm builds a model on the training data to make predictions or decisions without needing the original training data anymore.



Model-based Vs. Instance-based Learning Cont.

- **Instance-based Learning:**
An algorithm uses the training instances (or examples) themselves to make predictions.
- It doesn't build an explicit model; instead, it **memorizes** the training instances.



K-Nearest Neighbors (KNN) Algorithm

Given a new, unseen instance, the KNN algorithm searches the training set for the k training examples that are closest to the point, and then returns the most common output value among them:

1. Compute the **distance** between the new instance and every other instance in the dataset.
 2. Sort the distances to determine the closest k instances.
 3. For **classification**: Return the **mode** (most common) class label among the k nearest neighbors.
 4. For **regression**: Return the **mean** of the responses of the k nearest neighbors.
-

Distance Metrics

The concept of "closeness" or "distance" is fundamental to the KNN algorithm. Different metrics can be used to calculate the distance between instances:

- **Euclidean Distance:** This is the most common metric. It is the "ordinary" straight-line distance between two points in Euclidean space

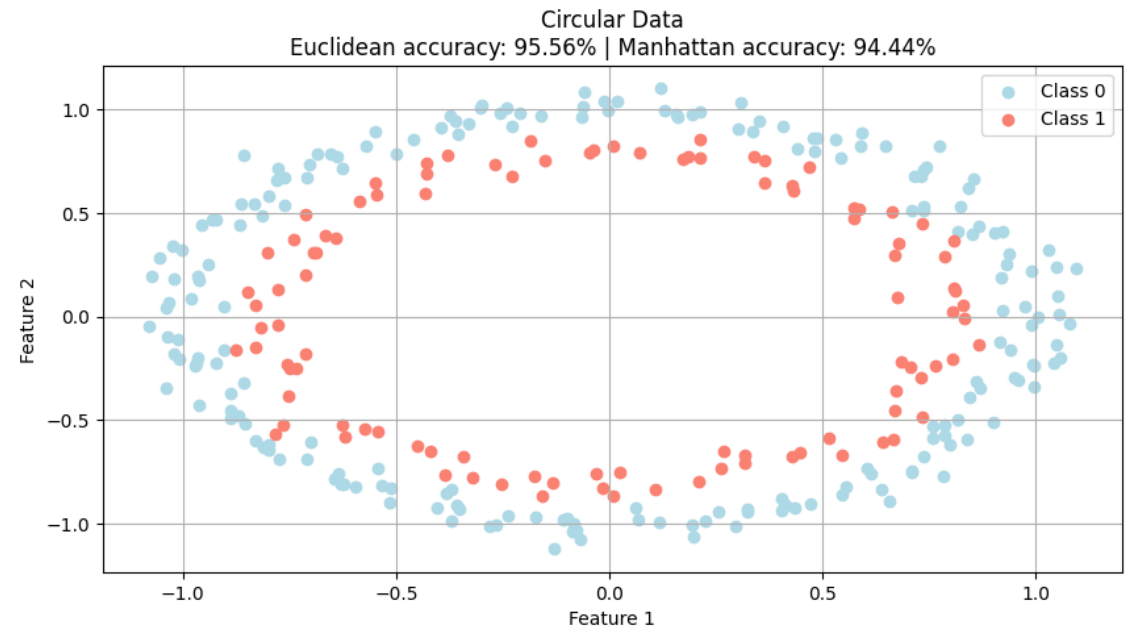
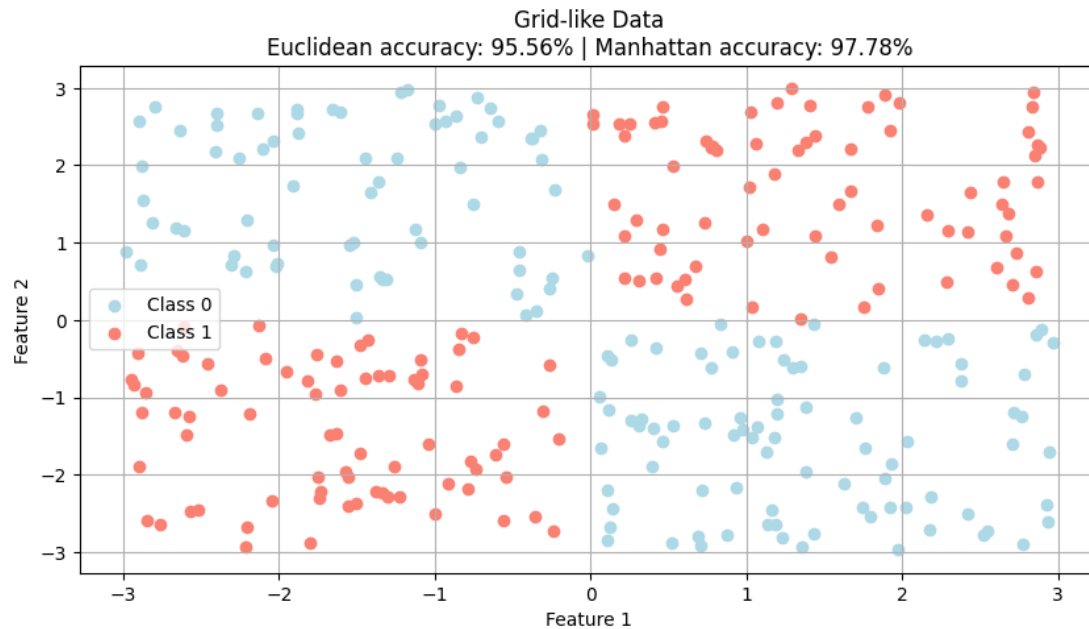
$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **Manhattan Distance:** Also known as the "city block" distance. It is the distance between two points measured along axes at right angles (like navigating the blocks of a grid-based city like Manhattan).

$$|x_2 - x_1| + |y_2 - y_1|$$

Choosing a Distance Metric

The distance metric determines how "closeness" is defined between instances in the feature space. The appropriateness of a distance metric depends on the nature of the data:



Advantages of KNN

Nonparametric:

- KNN does not make any explicit assumptions about the underlying distribution of data. This is especially valuable when the true distribution is unknown or non-standard.

Versatility:

- Can be used for both classification and regression.

Flexibility in Decision Boundaries:

- Unlike linear models that can only represent linear decision boundaries, KNN can capture more complex decision boundaries. A KNN decision boundary can bend to fit the data.
-

Disadvantages of KNN

Sensitive to Noisy Data:

- With KNN, outliers can significantly sway the model's predictions.

Computation Cost:

- For each prediction, KNN calculates the distance between the input instance and every other instance in the dataset.

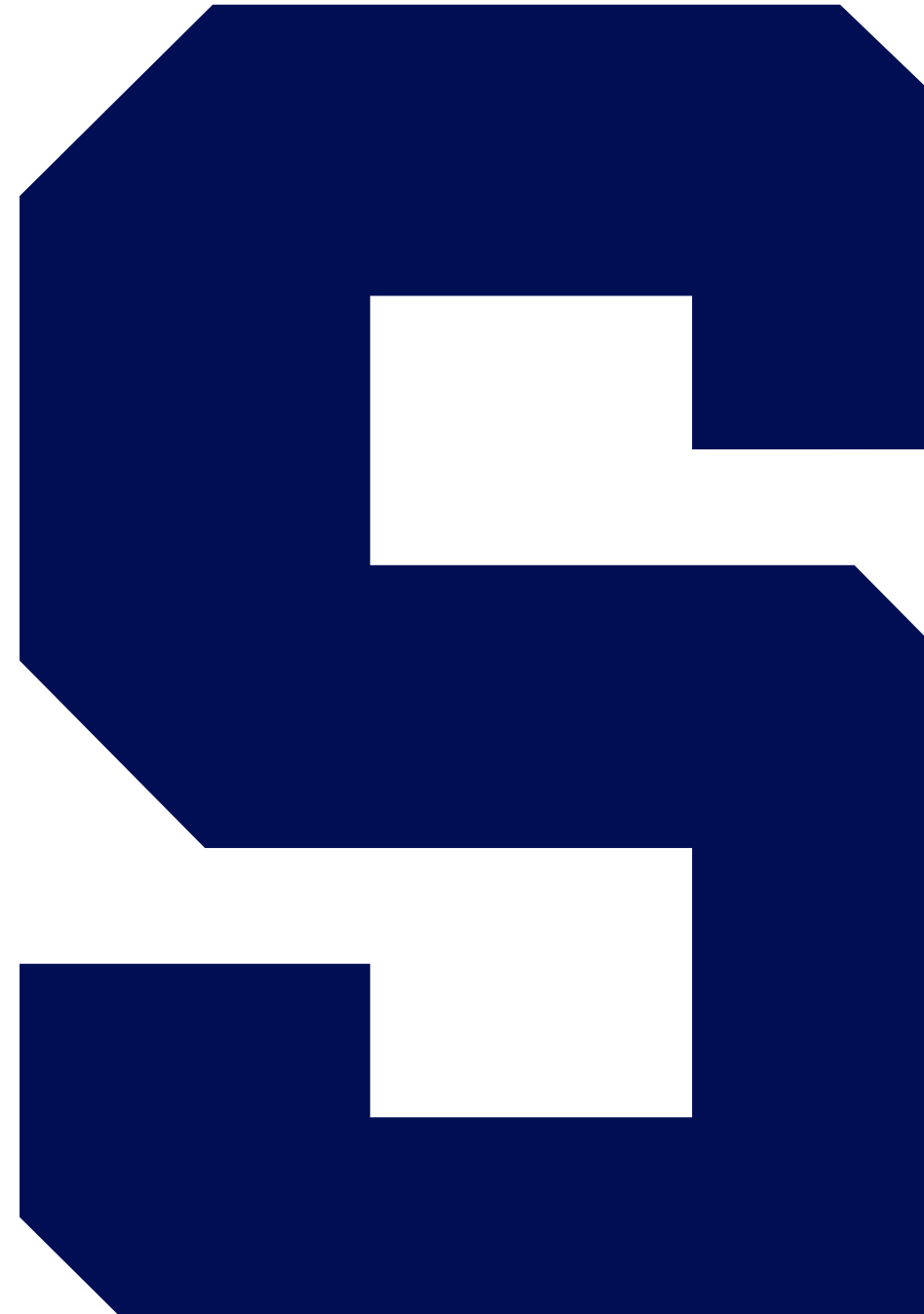
Memory Intensive:

- KNN is a lazy learner, meaning it doesn't explicitly learn a model during training. Instead, it memorizes the entire dataset.

Curse of Dimensionality:

- In high-dimensional spaces, the concept of "nearness" or "distance" becomes less distinct. Distances between instances tend to converge, making it hard for KNN to find meaningful neighbors which results in degraded performance.
-

Support Vector Machines

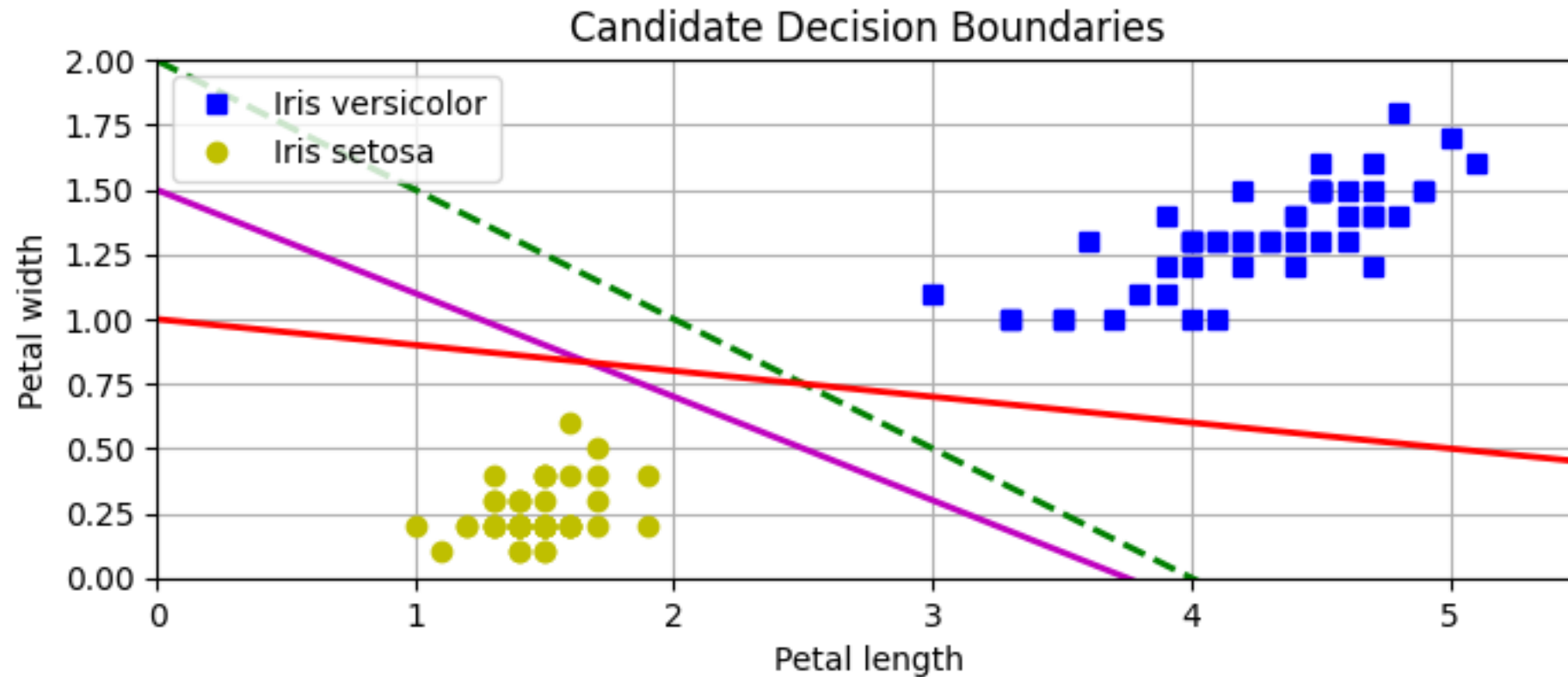


Support Vector Machines Background

- Idea originated in 1960s but fully emerged in 1990s.
 - Effective in classification and regression tasks.
 - Able to work with non-linearly separable data with “kernel trick.”
 - Use cases:
 - Image classification
 - Text classification
 - Bioinformatics
 - Financial forecasting
 - Anomaly detection
 - Alternative to more powerful neural networks when dataset is smaller and less complex.
-

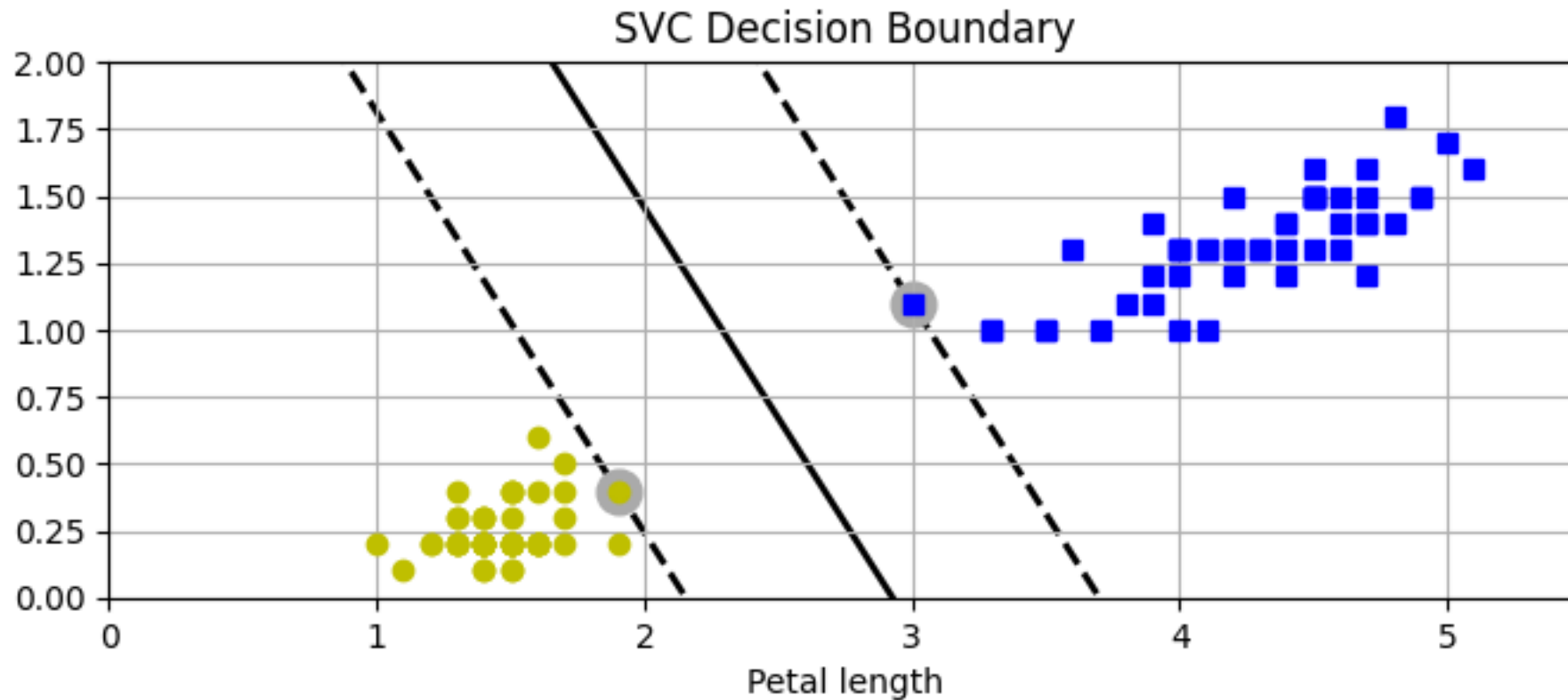
SVM Motivation

Choose the best decision boundary between two classes.



Choosing the Best Decision Boundary

Maximize the margin between the two classes.



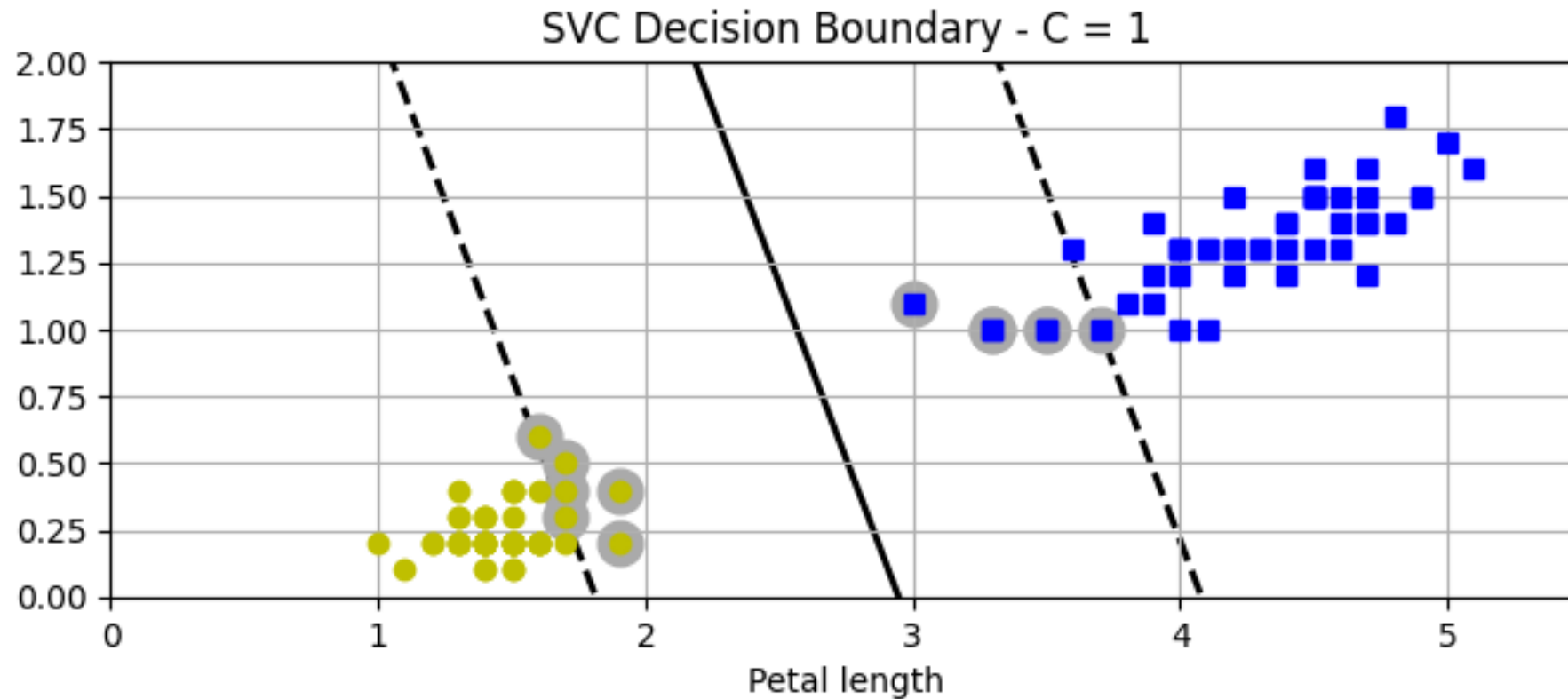
Hard Margin Vs. Soft Margin

Real-world data is often messy and may not be linearly separable.

- **Hard Margin SVM:** Assumes that the data is perfectly separable and tries to find a hyperplane with no violations (i.e., all points are on the correct side of the margin).
 - **Soft Margin SVM:** Allows some violations to occur, where a few points may be on the wrong side of the margin or even the decision boundary. This is where the regularization parameter, C , comes in. A **smaller C** allows more violations but can **generalize better**, while a **larger C** tries to classify all training examples correctly, **risking overfitting**.
-

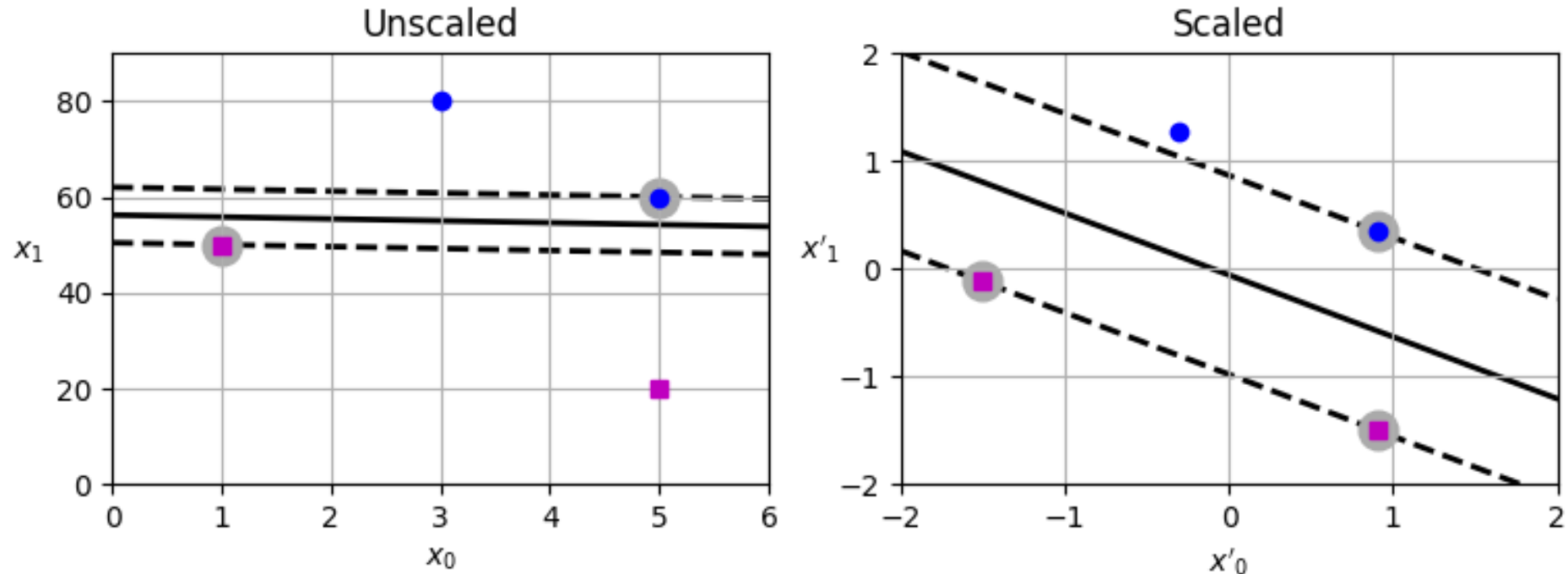
Hard Margin Vs. Soft Margin Cont.

Use the hyperparameter C to control regularization.



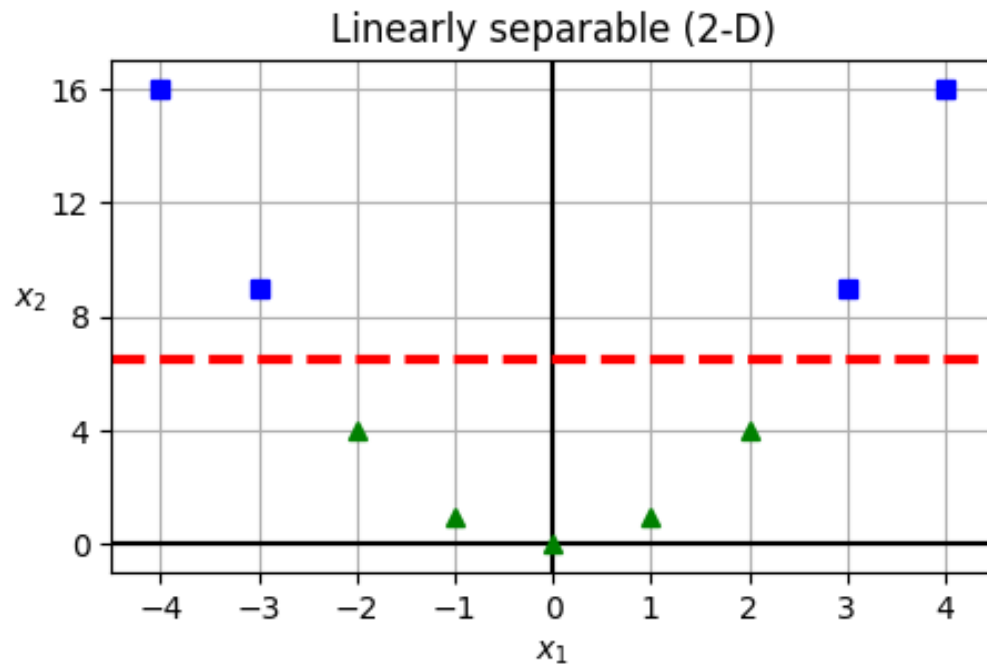
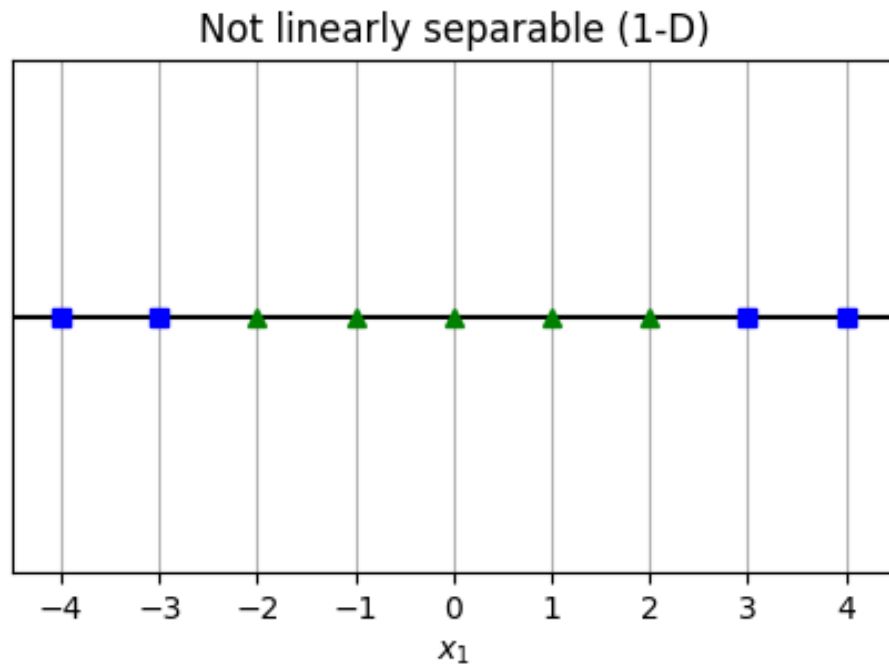
The Importance of Scaling

SVMs, at their core, are distance-based algorithms. The primary objective of SVM is to find a hyperplane that maximizes the margin between two classes. The computation of this margin is inherently dependent on the scale or units of the features.



Non-linearly Separable Data

Kernel transformations bring data into new, higher-dimensional space, where the data become linearly separable.



Common Kernels Used in SVMs

- **Linear kernel:** $K(x, y) = x \cdot y$
 - Use if the dataset is linearly separable or nearly linearly separable. It's less computationally expensive than other kernels.
 - **Polynomial kernel:** $K(x, y) = (1 + x \cdot y)^d$
 - Use if there's a reason to believe that the data could be separated not just by a line, but by some polynomial function. Increasing the degree of the polynomial increases the risk of overfitting.
 - **Radial Basis Function (RBF) or Gaussian kernel:** $K(x, y) = e^{-\gamma \|x-y\|^2}$
 - Useful for non-linear problems. When there's no prior knowledge about the data, the RBF kernel is a good default choice. Has an additional hyperparameter (gamma) to tune. If gamma is set too high, the decision boundary might be too strict and can overfit. If set too low, it might be too flexible and can underfit.
-

General Guidelines

- **Feature scaling:** Normalize or standardize before applying SVM with a kernel.
 - **Cross-validation:** Critical to use cross-validation to tune hyperparameters.
 - **Visual inspection:** Visualizing data in lower dimensions can help determine whether the decision boundary is linearly separable.
 - **Start simple:** Start with a linear kernel, then move on to more complex kernels if performance is not adequate.
 - **Overfitting:** Complex kernels are prone to overfitting.
 - **Computational resources:** Training an SVM with a complex kernel on a large dataset can be computationally intensive. Consider the available resources and the size of the dataset when choosing a kernel.
-